AWS Cloud Infrastructure Architecture Design Report

Educational Content Web Service — From Architecture Design to Operations Automation

Project	Educational Content Web Service — AWS Infrastructure Design
Role	Infrastructure Architect (Cloud Architecture Design)
Final Version	v4.5 (NAT Failover + Connectivity Canary Automation)
Project Duration	December 2025 – January 2026 (approx. 2 weeks)
Date	February 21, 2026
Region	ap-northeast-2 (Seoul)

1. Project Overview

1.1 Background & Objectives

This report documents a 4-person team project focused on designing and building AWS cloud infrastructure for an educational content web service. Rather than implementing web features, the primary objective was to validate practical capabilities in AWS infrastructure design, security, CI/CD pipeline construction, Infrastructure as Code (IaC), and operations automation. Application code was minimally implemented using AI-assisted development, allowing the team to concentrate on architectural decision-making and collaborative engineering processes.

Scope: architecture design, IaC drafting/specification, and decision documentation (implementation executed by other team members).

1.2 Team Composition & Role Assignment

Member	Role
Infrastructure Architect (Design Lead)	Infrastructure Architect — AWS architecture design and CloudFormation YAML authoring (design deliverables)
Team Member	Infrastructure Engineer — AWS stack deployment and build-phase troubleshooting
Team Member	Security Engineer — IAM, network security, and access control design & implementation
Team Member	Application Dev & CI/CD Lead — Application development, CI/CD implementation, WBS management

This report focuses on the work performed as the Infrastructure Architect, covering architectural decision-making and CloudFormation IaC drafting/specification (design deliverables), with deployment feedback incorporated through collaboration. A key theme is the iterative design refinement process driven by real-world deployment feedback from the infrastructure engineer.

1.3 Technology Stack

Cloud Platform	AWS (ap-northeast-2, Seoul)
IaC Tool	AWS CloudFormation (YAML)
Application Runtime	Python 3.11 / Django / Gunicorn
Web Server	Nginx (Reverse Proxy)
Database	Amazon RDS MariaDB (Multi-AZ)
CI/CD	GitHub Actions + CodeDeploy (team impl.) / CodePipeline design provided
Secret Management	AWS SSM Parameter Store (SecureString + KMS)
Instance Access	AWS SSM Session Manager (SSH-less operations)
OS / Instance	Ubuntu 22.04 LTS / t3.micro

2. Architectural Decision-Making Process

The infrastructure design was not finalized in a single pass. It evolved through a continuous cycle of initial design → deployment feedback → design revision → automation enhancement. This section documents that evolution and the reasoning behind each key decision.

2.1 Initial Design (v1–v2.1) — Cost Optimization & Maximum Security

Initial Design Goals

- Cost optimization: Eliminate NAT Gateway entirely; use VPC Endpoints exclusively for AWS service access
- Security: Block all internet egress from Private subnets at the network level
- High availability: 2-AZ subnet distribution + RDS Multi-AZ
- Reproducibility: CloudFormation stack enables repeatable create/delete cycles

Removing the NAT Gateway was driven not only by cost savings but also by a deliberate security design choice. Blocking outbound internet access from Private subnets eliminates potential C2 (command-and-control) communication paths and data exfiltration channels. Instead, AWS service access was restricted to an explicit allowlist via VPC Endpoints, following the principle of least privilege at the network level.

Additionally, while Amazon ElastiCache was initially considered for Redis caching, it was determined that the cost and operational complexity outweighed the benefits for a 2-week demo project.

2.2 Real-World Constraints — Feedback from Deployment

Issues Encountered During Deployment

① Package download failures (apt/pip) due to blocked internet egress
② CodeDeploy agent installation instability (agent downloaded from AWS S3 regional endpoint)
③ Unreliable bootstrap success rate in a NAT-less environment
→ Issues identified through iterative feedback from the infrastructure engineer during actual AWS deployment

As the infrastructure engineer attempted to deploy using my CloudFormation YAML, the limitations of a NAT-less environment became apparent. While theoretically sufficient, the S3 Gateway Endpoint and SSM Interface Endpoints did not fully cover all egress paths required during the CodeDeploy agent bootstrap phase, leading to intermittent failures.

This feedback prompted a design revision. The key principle was not to abandon security goals, but to shift from 'complete blocking' to 'minimum necessary allowance' — finding the right balance point.

2.3 Design Revision (v3.x–v4.3) — Incorporating Deployment Reality

Core Revision Principles

- Shift from 'full NAT removal' to 'single NAT (AZ-A) + Endpoint bypass' for balanced approach
- AWS service traffic (S3, SSM) still routed via Endpoints — minimizing NAT usage and exposure
- ElastiCache removed → Local Redis adopted (right-sized for demo scope)
- CI/CD pipeline integrated into infrastructure.yaml → entire environment reproducible from one stack

Minimizing to a single NAT Gateway (AZ-A only) reduced cost while restoring reliable bootstrap for external package downloads. S3 and SSM traffic continued to route via VPC Endpoints, keeping NAT-exposed internet traffic to a minimum. The YAML file evolved rapidly from v3.0 through v4.3, with each version incorporating specific feedback: staged deployment hook separation, Gunicorn systemd unit addition, SSM environment variable injection improvements, and more.

2.4 Final Design Enhancement (v4.5) — Operations Automation

While a single NAT Gateway is cost-efficient, it creates a single point of failure (SPOF): if AZ-A goes down, outbound internet access for both Private App subnets is affected. To mitigate this, v4.5 introduced Lambda-based automatic failover.

v4.5 Automation — NAT Failover + Connectivity Canary

① Connectivity Canary Lambda: Periodically tests outbound connectivity from Private subnets
→ Proactively detects NAT issues before user impact occurs
② CloudWatch Alarm: Monitors NAT ErrorPortAllocation / PacketsDropCount or Canary failures
③ EventBridge: Receives alarm state change events and triggers the Failover Lambda
④ NAT Failover Lambda: Automatically creates a Failover NAT Gateway in AZ-C Public Subnet and updates Private App route tables' default route (0.0.0.0/0) to the Failover NAT
⑤ Auto-rollback: When alarm returns to OK, restores original route (NAT-A) and deletes Failover NAT/EIP

The NatGatewayMode parameter allows selecting between single (default, cost-optimized) and per-az (high availability) modes, providing flexibility based on the deployment context — demo environments vs. higher-availability setups.

Parameter	Default	Description
NatGatewayMode	single	single: NAT in AZ-A only (cost-optimized) / per-az: NAT per AZ (high availability)
EnableNatAutoFailover	true	Enable Lambda-based automatic NAT failover
EnableNatConnectivityCanary	true	Enable periodic outbound connectivity test Lambda
CreatePipeline	true	Include CodePipeline/CodeBuild CI/CD resources in stack

3. Design Version History

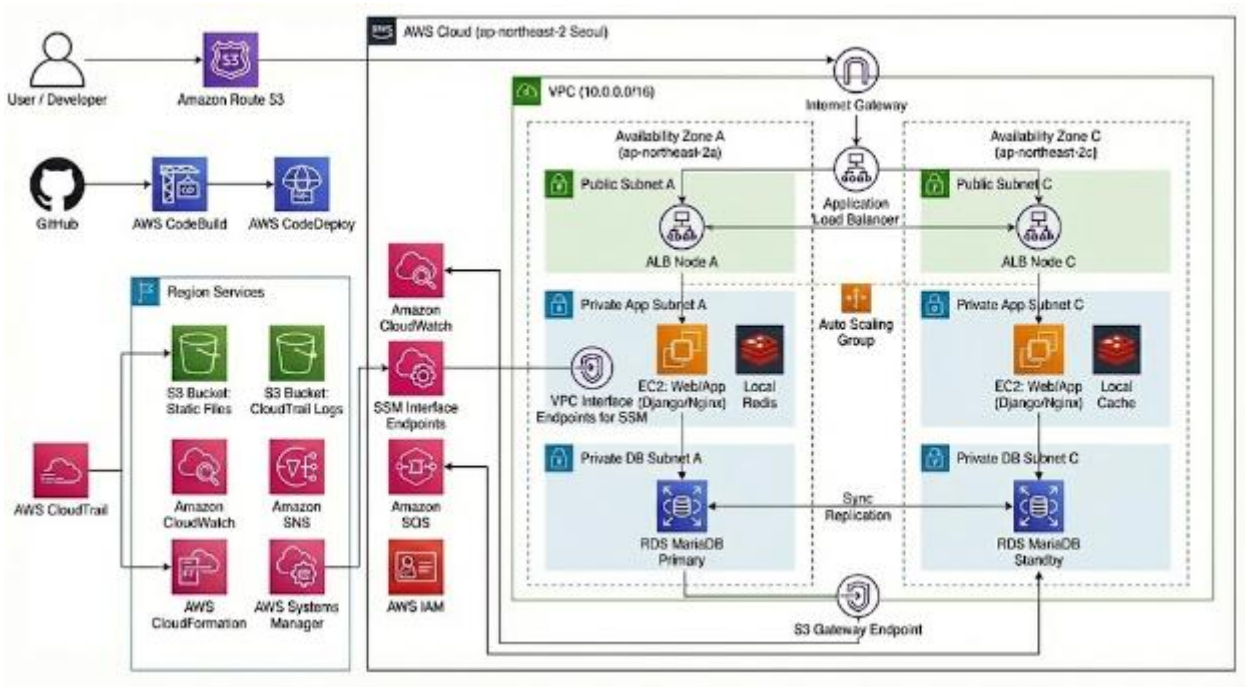
The following table captures the full evolution of the CloudFormation YAML as deployment feedback was incorporated. Each version reflects a concrete architectural decision.

Version	Date	Key Changes & Design Intent
v1 (edu_v1)	Dec 23, 2025	Base VPC + ALB + ASG + RDS. No NAT — Endpoint-only Private connectivity attempted
v2.1	Dec 24, 2025	CI/CD pipeline extracted to pipeline.yaml; SSM env injection scripts added
v3.0	Dec 31, 2025	Deployment feedback incorporated: NAT Gateway (single) introduced. Golden AMI baking adopted
v3.1–3.4	Dec 31, 2025	Deployment hooks broken into stages (00–04); Gunicorn systemd unit added; AMI scripts refined
v3.5–3.6	Dec 31, 2025	00_inject_env_from_ssm.sh added (SSM → .env automation); Redis AMI separated
v3.8	Dec 31, 2025	infrastructure.yaml significantly expanded (27KB); consolidated and stabilized
v4.3	Jan 5, 2026	Unified single-stack: Nginx config + systemd units packaged; CI/CD pipeline integrated
v4.5 (Final)	Jan 5, 2026	NAT Failover Lambda + Connectivity Canary automation added; operations automation complete

4. Final Architecture Detail

4.1 Architecture Overview

The final architecture (v4.5) spans two Availability Zones (AZ-A and AZ-C) with a 3-tier layout: Public / Private App / Private DB. The core design achievement is cost efficiency through a single NAT Gateway, combined with Lambda-based failover automation to maintain operational resilience.



▲ *Final AWS Infrastructure Architecture Diagram (2-AZ, 3-Tier)*

4.2 Network Layout

Subnet	CIDR	AZ	Purpose
Public Subnet A	10.0.0.0/24	ap-northeast-2a	ALB Node A, NAT Gateway A (Primary)
Public Subnet C	10.0.1.0/24	ap-northeast-2c	ALB Node C, Failover NAT (auto-created on failure)
Private App Subnet A	10.0.10.0/24	ap-northeast-2a	EC2 Web/App instances (Django + Nginx + Redis)
Private App Subnet C	10.0.11.0/24	ap-northeast-2c	EC2 Web/App instances (Django + Nginx + Local Cache)
Private DB Subnet A	10.0.20.0/24	ap-northeast-2a	RDS MariaDB Primary
Private DB Subnet C	10.0.21.0/24	ap-northeast-2c	RDS MariaDB Standby

4.3 Security Group Design

Security groups are designed following the principle of least privilege, referencing security group IDs as sources (rather than IP ranges) to maintain precise access control in dynamic auto-scaling environments.

Security Group	Inbound	Source	Purpose
AlbSG	TCP 80	0.0.0.0/0	Internet → ALB
AppSG	TCP 80	AlbSG	ALB → EC2 app servers
DbSG	TCP 3306	AppSG	EC2 → RDS MariaDB
VpcEndpointSG	TCP 443	VPC CIDR	EC2 → SSM/S3 Interface Endpoints
RedisSG	TCP 6379	AppSG	EC2 app servers → Local Redis EC2

4.4 VPC Endpoints

To minimize internet exposure, all AWS managed service access is handled through VPC Endpoints:

- S3 Gateway Endpoint: Routes S3 traffic (artifacts, static files) through the VPC internal path — no internet exposure, no data transfer cost
- SSM Interface Endpoints (ssm, ec2messages, ssmmessages): Enables Session Manager access to instances without SSH, keeping operations fully within the VPC

5. Key Components

5.1 Application Load Balancer (ALB)

- Internet-facing ALB spanning 2 AZs in Public subnets
- HTTP (port 80) listener → forwards to Private App subnet EC2 Target Group
- Health check: GET /healthz (HTTP 200–399, 15-second interval, threshold: healthy 2 / unhealthy 3)
- Nginx configured to respond 200 on /healthz immediately post-launch → ALB health check can pass even before app startup

5.2 EC2 Auto Scaling Group (ASG)

- Launch Template: Ubuntu 22.04 LTS, t3.micro, Nginx + CodeDeploy Agent auto-installed via UserData
- Desired: 2 / Min: 2 / Max: 4 (evenly distributed across 2 AZs)
- ELB health check integration (GracePeriod: 180 seconds)
- IAM Instance Profile: SSM Session Manager access + SSM Parameter Store read + S3 artifact access

5.3 RDS MariaDB (Multi-AZ)

- Engine: MariaDB / Instance: db.t3.micro / Storage: 20GB gp2
- Multi-AZ: Primary (AZ-A) + Standby (AZ-C) with automatic failover
- Placed in Private DB subnets — no public accessibility
- All connection details stored in SSM Parameter Store (no hardcoded credentials)

5.4 SSM Parameter Store — Environment Variable Management

SSM Parameter Path	Type	Purpose
/{prefix}/db/host	String	RDS Endpoint — auto-injected from CloudFormation Output
/{prefix}/db/name	String	Database name
/{prefix}/db/user	String	Database user
/{prefix}/db/password	SecureString	Database password (KMS encrypted)
/{prefix}/django/secret_key	SecureString	Django SECRET_KEY (KMS encrypted)
/{prefix}/django/allowed_hosts	String	Django ALLOWED_HOSTS
/{prefix}/django/debug	String	Django DEBUG flag
/{prefix}/app/static_bucket	String	Static files S3 bucket name

6. CI/CD Pipeline

6.1 Designed vs. Implemented

As the infrastructure architect, I designed and provided specifications for a fully integrated CI/CD pipeline using AWS CodePipeline (embedded in the CloudFormation stack). The team ultimately adopted a GitHub Actions + CodeDeploy approach for implementation. Both methods share the same CodeDeploy deployment hook structure.

	Designed (CloudFormation)	Implemented (GitHub Actions)
Trigger	GitHub → CodeStar Connection → CodePipeline	GitHub push → GitHub Actions Workflow
Build	CodeBuild (buildspec.yml)	GitHub Actions → AWS CLI → S3 upload
Deploy	CodeDeploy (appspec.yml)	CodeDeploy (same appspec.yml)
Advantage	Full environment reproduced from single stack	Simpler setup, unified management in GitHub

6.2 CodeDeploy Deployment Hooks

The lifecycle hooks defined in appspec.yml were designed as part of the architecture and aligned across both CI/CD approaches.

Lifecycle Event	Script	Action
ApplicationStop	10_stop_services.sh	Stop running Gunicorn and Nginx services
BeforeInstall	01_prepare_dirs.sh	Create deployment directories and set permissions
AfterInstall	02_install_runtime_files.sh	Install Nginx config and systemd unit files
ApplicationStart	03_start_services.sh	Inject .env from SSM, then start Gunicorn and Nginx
ValidateService	04_validate_service.sh	Validate deployment via curl to /healthz endpoint

7. Observability

7.1 CloudWatch Alarms

Alarm	Metric	Threshold	Purpose
{prefix}-alb-target-5xx	HTTPCode_Target_5XX_Count	≥ 1 / 1 min	Immediate application error detection
{prefix}-asg-inservice-low	GroupInServiceInstances	< 1 (2 consecutive)	Total instance failure detection
NatConnectivityAlarm	Lambda Canary errors	Canary failure	Proactive NAT/egress issue detection

7.2 CloudTrail Audit Logging

All AWS API calls are captured by CloudTrail and stored in a dedicated S3 bucket with a bucket policy restricting write access to the CloudTrail service only. Multi-region events and global service events are included for comprehensive audit coverage.

7.3 SNS Alerting

CloudWatch alarms are routed to an SNS Topic for immediate email notifications. NAT Failover Lambda execution also triggers SNS notifications, giving operators full visibility into failure detection, automated remediation, and recovery status.

8. Design Comparison: Initial vs. Final

Item	Initial Design (v1–v2.1)	Final Design (v4.5)
NAT Gateway	None (fully removed for cost & security)	1 instance (AZ-A, single mode by default)
Internet Egress	Fully blocked → Endpoint-only allowlist	Minimum allowance (NAT) + Endpoint bypass
Redis / Cache	Amazon ElastiCache (under consideration)	Local Redis (simplified, cost-reduced)
CI/CD Stack	Separate pipeline YAML	Integrated into infrastructure.yaml (single stack)
Failure Response	Manual intervention	Lambda Failover + Canary automation
Parameterization	Basic configuration	NatGatewayMode, EnableNatAutoFailover selectable
YAML Size	~18KB	~46KB (significantly expanded)

9. Conclusion & Contribution Summary

9.1 Key Contributions as Infrastructure Architect

- Designed the full AWS infrastructure architecture: 3-tier VPC, subnets, security groups, and VPC Endpoint configuration
- Authored the complete CloudFormation infrastructure.yaml from scratch (v1 → v4.5, 8 versions over ~2 weeks)
- Iteratively refined the design by incorporating real deployment feedback (NAT reintroduction, deployment script restructuring)
- Designed and documented v4.5 NAT Failover Lambda + Connectivity Canary automation; provided specification to build engineer for implementation
- Designed CodeDeploy lifecycle hook structure (ApplicationStop → ValidateService) and deployment script specifications
- Established SSM Parameter Store-based environment variable management, eliminating hardcoded secrets
- Authored the presentation Q&A cheat sheet — documenting the rationale behind every architectural decision

9.2 Key Technical Learnings

This project provided practical experience translating cloud architecture concepts into reviewable IaC specifications. Most importantly, it reinforced that the best architecture is not the most elegant one on paper, but the one that works reliably in real deployments — which requires close collaboration and tight feedback loops with the engineers executing the deployment.

The NAT Gateway decision process — balancing cost, reliability, and security in a triangle of trade-offs — is a challenge that appears constantly in real-world cloud architecture. Arriving at the final solution (single NAT + Endpoint bypass + Lambda Failover automation) through iterative refinement was a practical demonstration of pragmatic systems design.

9.3 Future Improvements

- Evaluate NAT per-az mode or self-managed NAT instances for better cost/HA trade-off
- Adopt Blue/Green deployment strategy to improve deployment safety (currently In-Place)
- Introduce Amazon ElastiCache for managed Redis with improved reliability
- Add CloudFront + WAF for CDN performance and web application firewall protection
- Standardize to role-based temporary credentials (eliminate long-lived IAM access keys)